Reviewer Pack — Coxeter Group Enumeration of Boolean Function Entropy via In…

Entry e8bee6ae084a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 23:30:29 UTC

Coxeter Group Enumeration of Boolean Function Entropy via Invariant Generators

Entry ID: `e8bee6ae084a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-30 23:30:29 UTC

1. Conjecture

Field A (mathematical branch): Coxeter group theory **Field B** (complexity object): Boolean function entropy

Statement:

> For a given boolean function f with n variables, the number of invariant generators of its Coxeter group action, when counted using a specific polynomial representation, is Θ(n^(1/2) log n). Specifically, |I(f)| = Θ(n^(1/2) log n), where I(f) represents the set of invariant generators for the function f.

Rationale (proposer's reasoning):

> The connection between Coxeter group theory and boolean function entropy could expose a new structural insight into the complexity of boolean functions. Coxeter groups provide a framework to study symmetry in mathematical structures, which might be utilized to characterize the inherent complexity of evaluating boolean functions.

Taxonomy category: `Coxeter_group_action` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d50a94545b6412a7`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the correlation coefficient between the number of invariant generators |I(f)| and n^(1/2) log n for a set of at least 30 random boolean functions with n ≤ 40 is ≥ 0.8, and falsified if this correlation coefficient is < 0.5.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Coxeter group theory" AND "Boolean function entropy" AND "polynomial representation"` - `"invariant generators" IN COXETER group AND boolean function entropy` - `"number of generators" IN Coxeter group AND Boolean function entropy`

Top relevant hits considered: - [http://arxiv.org/abs/1405.3051v1] Involution Products in Coxeter Groups - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/1911.04516v1] Boolean lattices in finite alternating and symmetric groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def evaluate_polynomial(polynomial, variables):
        result = polynomial[0]
        for i, coeff in enumerate(polynomial[1:], start=1):
            result += coeff * variables[i-1]
        return int(result % 2)

    def is_invariant(polynomial, function):
        n = len(function)
        for i in range(2**n):
            variables = [int(x) for x in format(i, f'0{n}b')]
            if evaluate_polynomial(polynomial, variables) != evaluate_polynomial(polynomial, [(1 -
                return False
        return True

    def generate_coxeter_group_invariant_generators(n):
        generators = []
        for i in range(1, n):
            polynomial = [0] * (n + 1)
            polynomial[i] = 1
```

```
            if is_invariant(polynomial, generate_boolean_function(n)):
                generators.append(polynomial)
        return generators

    def correlation_coefficient(x, y):
        mean_x = sum(x) / len(x)
        mean_y = sum(y) / len(y)
        numerator = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(len(x)))
        denominator = math.sqrt(sum((x[i] - mean_x)**2 for i in range(len(x))) * sum((y[i] - mean_
        return numerator / denominator if denominator != 0 else 0

    n_values = [5, 10, 15, 20, 30, 40]
    invariant_counts = []

    for n in n_values:
        for _ in range(5):
            function = generate_boolean_function(n)
            generators = generate_coxeter_group_invariant_generators(n)
            invariant_counts.append(len(generators))

    n_max = max(n_values)
    instances_tested = len(invariant_counts)
    metric_value = correlation_coefficient(range(1, n_max + 1), invariant_counts)
    conjecture_holds = metric_value >= 0.8
    counterexample = "" if conjecture_holds else f"Correlation coefficient {metric_value:.2f} < 0.

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value:.2f} std=0.00 support_fraction=1.00")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.8\" first_failing_s
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it was unable to complete the required analysis of at least 30 random boolean functions with n ≤ 40 to determine if the correlation coefficient meets the support condition. | next: Run the test again ensuring it completes without crashing or timing out. If successful, re-evaluate the correlation coefficient between |I(f)| and n^(1/2) log n for at least 30 random boolean functions with n ≤ 40 to determine if the conjecture is supported.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23707
2	preregistration	ollama_remote	glm4:latest	0	0	11947
3	novelty	ollama_remote	glm4:latest	0	0	8563
4	novelty	ollama_remote	glm4:latest	0	0	9674
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14462
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10555
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17988
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44457
9	judge	ollama_remote	glm4:latest	0	0	42808

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 184161 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e8bee6ae084a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e8bee6ae084a.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e8bee6ae084a.tar.gz` (if generated)